

16-BIT LOW POWER CMOS MULTIPLIER IC DESIGN

Nicholas Ding Zhuleong

School of Electrical & Electronic Engineering
Nanyang Technological University
Nanyang Avenue, Singapore 639798
e-mail: e240212@e.ntu.edu.sg

Supervised by: A/P Gwee Bah Hwee

Keywords: CMOS, Integrated IC, Low power, Multiplier architecture, PPA

Abstract

Arithmetic circuits such as digital adders and multipliers are fundamental components in digital signal processors (DSPs), particularly for filtering applications in modern wireless devices where low power consumption is critical. To address the increasing demand for energy efficient circuits in portable systems, this work presents the design and implementation of a 16-bit low-power CMOS multiplier.

The proposed multiplier architecture employs a modified radix-4 Booth encoder in conjunction with a Dadda reduction tree. The Booth encoding reduces the number of partial products, while the Dadda tree minimises the number of reduction stages, lowering propagation delay and improving computational efficiency.

The design is implemented using a GF 22nm CMOS process and evaluated using Synopsys IC design tools. For comparison, a conventional 16-bit array multiplier is also designed under similar conditions. Results show that the proposed multiplier achieves a total dynamic power consumption of $42.23\mu\text{W}$, representing a 32.23% reduction compared to the array multiplier. Additionally, the propagation delay is reduced to 1.62ns, yielding a 33.88% improvement in speed.

However, these gains come at the cost of increased cell area, with the proposed design occupying 930 units compared to 530 units for the array multiplier where 1 unit is $1\mu\text{m}^2$. Despite this trade off, the improved energy delay performance demonstrates that the proposed Booth Dadda architecture is a highly effective solution for high performance, low power DSP applications in next generation wireless devices.

1 Introduction

1.1 Background

Modern wireless devices and portable digital signal processors (DSPs) increasingly require high performance arithmetic units that operate within strict power constraints. Among these units, multipliers are fundamental components used in computation intensive applications such as digital filtering, image processing and artificial

neural networks [1]. This is because multiplication is frequently performed within the arithmetic logic unit (ALU), multiplier circuits often become a primary bottleneck in terms of both system throughput and power consumption. As a result, the design of energy efficient multipliers is critical for improving the overall performance and longevity of modern VLSI systems [2].

Over the years, several techniques have been developed to address the high computational cost of multiplication. Booth encoding, introduced by Booth in 1951, reduces the number of partial products by recoding the multiplier, decreasing the number of required addition operations [3]. To further improve efficiency, tree based reduction structures are commonly used to compress these partial products, while Wallace trees are widely adopted for their speed. Dadda reduction tree, proposed in 1965, offers a more structured approach that can reduce propagation delay and optimise hardware usage in the later stages of reduction [4].

Despite these advancements, designing multipliers in modern submicron CMOS technologies still involves a trade off between power consumption, delay and area. Conventional array multipliers are attractive due to their regular structure and relatively small area footprint; however, they suffer from longer propagation delays and increased switching activity, leading to higher dynamic power consumption [5]. These limitations highlight the need for more optimised architectures that can better balance performance and energy efficiency.

1.2 Objectives and Scopes

In this work, a 16-bit low power CMOS multiplier is designed and implemented using a modified radix-4 Booth encoder combined with a Dadda reduction tree. The objective is to reduce dynamic power consumption and propagation delay while maintaining a reasonable area overhead. The proposed design is implemented using a GF 22 nm CMOS process and evaluated with Synopsys IC design tools. Its performance is compared against a conventional 16-bit array multiplier under identical conditions to assess improvements in power, propagation delay and overall efficiency.

1.3 Organisation of the Report

The remainder of this paper is outlined as follows: In section 2, we explore the different architectures and optimising strategies used in previous works in designing a multiplier.

In section 3, we explore the design methodology of the optimised multiplier. In section 4, we simulate and evaluate the performance of the optimised multiplier compared to conventional array multiplier. Lastly, we conclude the work in section 5.

2 Literature Review

2.1 Binary Multiplications

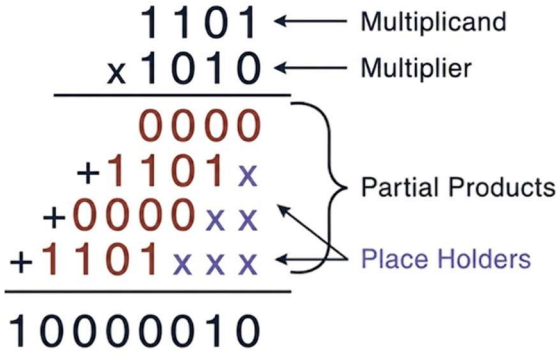


Figure 1: Binary Multiplication of 1101 and 1010.

Binary multiplication is a fundamental arithmetic operation in digital electronics and computer science. Conceptually, it is remarkably similar to the long multiplication method we learn in primary school for base-10, decimal numbers. The process is actually much simpler due to the binary numeral system operates in base-2. A binary digit or bit, can only be a '0' or a '1'. This means that when you multiply the multiplicand by a single bit of the multiplier, the result is always either zero or an exact copy of the multiplicand itself.

There are no complex multiplication tables to memorise in binary. The fundamental rules are straightforward:

- 0 x 0 = 0 (1)
- 0 x 1 = 0 (2)
- 1 x 0 = 0 (3)
- 1 x 1 = 1 (4)

To multiply two binary numbers, you look at the multiplier one bit at a time, starting from the rightmost bit which is also the least significant bit. If the multiplier bit is a 1, you write down the multiplicand. If the multiplier bit is a 0, you write down a row of zeros.

For each subsequent bit of the multiplier as you move to the left, you shift the resulting row which is called a partial product one position to the left, adding place holders from left. Once you have generated a partial product for every bit in the multiplier, you simply add all these shifted rows together using binary addition to get the final product.

In the context of the hardware design we were discussing earlier, this mathematical concept can be translated directly into logic gates. Generating the many rows of partial products is done almost instantly using a parallel array of AND gates.

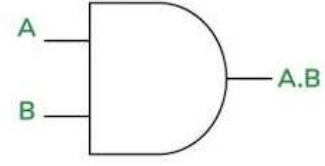


Figure 2: Gate Level Schematic of an AND gate.

Table 1: Truth Table of a AND gate.

A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

2.2 Adder Units

The accumulation of partial products represents the most delay intensive phase of the multiplication process. Consequently, the fundamental logic blocks responsible for this addition namely the Half Adder (HA) and Full Adder (FA) dictate the foundational power and speed characteristics of the entire multiplier circuit [6].

2.2.1 Half Adders (HA)

The Half Adder is the simplest arithmetic combinational logic circuit, designed to compute the addition of two single-bit binary inputs, A and B. The operation produces two outputs: the Sum (S) and the Carry (C). At the logic gate level, the Sum is generated using an Exclusive-OR (XOR) gate, while the Carry is generated using a standard AND gate. The Boolean expressions governing the Half Adder are defined as:

$$S = A \oplus B \quad (5)$$

$$C = A \cdot B \quad (6)$$

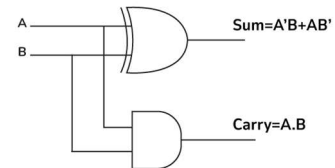


Figure 3: Gate Level Schematic of a Half Adder.

Table 2: Truth Table of a HA.

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

While highly efficient in terms of transistor count, the Half Adder is limited in its application within large arithmetic arrays because it lacks the capacity to accept a carry bit from a previous lower order addition stage.

2.2.2 Full Adder (FA)

To resolve the limitations of the HA, the Full Adder is employed. A Full Adder computes the sum of three binary bits: two significant operand bits of A and B and an incoming carry bit (C_{in}) from a preceding, less significant operational stage [5]. This capability allows FAs to be cascaded to perform multi-bit addition. The FA logic generates a Sum (S) and a Carry-Out (C_{out}) defined by the following Boolean equations:

$$S = A \oplus B \oplus C_{in} \quad (7)$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B)) \quad (8)$$

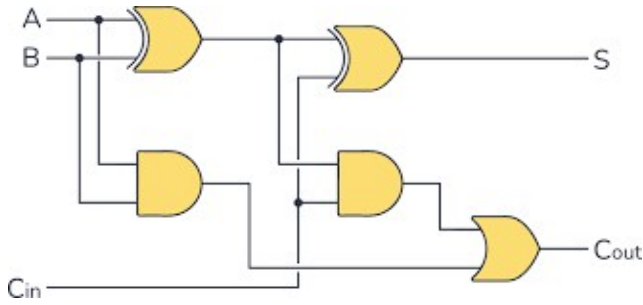


Figure 4: Gate Level Schematic of a Full Adder.

Table 3: Truth Table of a FA.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

In conventional multiplier architectures, Full Adders are tiled in a highly regular two dimensional grid to form an array. However, because the C_{out} of one FA serves as the C_{in} of the next contiguous stage, the propagation delay of the circuit increases linearly with the bit-width of the operands [7]. This linear delay dependency, commonly referred to as the carry propagation chain, creates a severe performance bottleneck and induces significant dynamic power dissipation due to signal glitching, necessitating the use of more advanced parallel reduction architectures.

2.3 Conventional Array Multiplier

The conventional array multiplier represents the most direct hardware implementation of the standard mathematical shift and add multiplication algorithm. In this architecture, the partial product matrix is generated simultaneously using a two dimensional parallel array of AND gates. Once generated, these partial products are accumulated using a highly structured grid of HA and FA [8].

2.3.1 Architectural Characteristics

In an $N \times N$ array multiplier, the accumulation matrix requires $N(N - 1)$ adders. The primary advantage of this topology is its structural regularity. This is because the adders are tiled in a predictable, repeating rectangular grid, the physical layout and routing processes during VLSI physical design are straightforward. This topological symmetry makes array multipliers highly modular and easy to scale for smaller bit-widths.

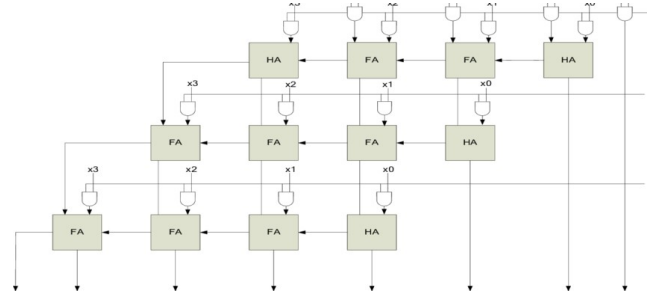


Figure 5: Block Diagram of a 4 x 4 Array Multiplier.

2.3.2. Performance Limitations and Glitching

Despite its ease of implementation, the standard array multiplier suffers from severe performance degradation as the operand bit-width scales up to 16 bits and beyond, making it suboptimal for high performance DSP applications in wireless devices.

The most critical limitation is its propagation delay. The critical path of an array multiplier traverses diagonally from the top right of the array to the bottom left, dictated by the continuous carry propagation chain. Consequently, the delay scales linearly, exhibiting an $O(N)$ time complexity.

Furthermore, this linear, multistage cascading structure is highly susceptible to unequal signal arrival times because signals travelling down the left side of the array pass through more logic gates than those on the right, the inputs to deeper FA do not arrive simultaneously. This structural disparity leads to spurious logic transitions, commonly known as glitching before the final signals settle into their steady state [9]. In CMOS circuits, every toggling event charges and discharges parasitic capacitances, meaning this glitching phenomenon translates directly into wasted dynamic power dissipation.

To mitigate the linear delay and high switching activity inherent to the array multiplier, modern low power datapaths must employ advanced algorithms to reduce the initial number of partial products and reorganise the accumulation matrix into a parallel, logarithmic delay profile.

2.4 Partial Product Generation: Booth Encoding

The primary bottleneck in a standard $N \times N$ multiplier is the generation and subsequent accumulation of N discrete

partial product rows. For a 16-bit array multiplier, 16 rows must be compressed, requiring a deep and power intensive adder tree. To optimise this process, algorithmic reduction of the initial partial product matrix is required.

2.4.1 Standard Booth Encoding

The original Booth algorithm, introduced by A. D. Booth in 1951, provides a method for multiplying signed binary numbers in two's complement notation [10]. The algorithm scans the multiplier bits sequentially and performs additions or subtractions only when a transition from 0 $\rightarrow$ 1 or 1 $\rightarrow$ 0 occurs, effectively skipping over contiguous strings of zeros or ones. While highly efficient for sequential processing, the standard Radix-2 Booth algorithm generates a variable number of partial products depending on the specific bit pattern of the multiplier. This variability complicates the design of parallel, synchronous hardware architectures.

2.4.2 Modified Radix-4 Booth Encoding (MBE)

To resolve the variable accumulation depth of the original algorithm, O. L. MacSorley introduced the Modified Booth Encoding (MBE), commonly referred to as Radix-4 Booth encoding [11]. This architecture guarantees a fixed reduction in the number of partial products by exactly half $N/2$.

The Radix-4 MBE algorithm achieves this by partitioning the multiplier into overlapping windows of three bits. Each 3-bit window consists of the current bit, the next most significant bit and the previous least significant bit (y_{i+1} , y_i , y_{i-1}). Based on the binary value of this window, the algorithm selects a specific multiple of the multiplicand X to form the partial product. The possible encoded operations are restricted to the set $\{-2X, -X, 0, +X, +2X\}$.

Table 4: Truth Table for MBE.

y_{i+1}	y_i	y_{i-1}	Partial Product
0	0	0	0
0	0	1	X
0	1	0	X
0	1	1	2X
1	0	0	-2X
1	0	1	-X
1	1	0	-X
1	1	1	0

2.4.3 Hardware Implementation Advantages

The operations required by the Radix-4 set are highly advantageous for CMOS hardware implementation because they do not require complex arithmetic.

- $\pm X$: Passed directly to the output or inverted.
- $\pm 2X$: Achieved simply by shifting the multiplicand done bit position to the left (a zero-cost physical wiring shift).
- **Negative multiples (-X, -2X)**: Computed by inverting the bits of the shifted or unshifted multiplicand and

adding a '1' to the least significant bit (two's complement).

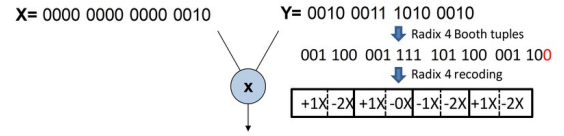


Figure 6: Example of Radix-4 Booth recoding.

By strictly halving the number of partial product rows (from 16 rows down to 8 rows for a 16-bit multiplier), the Radix-4 Booth encoder significantly reduces the capacitive load and the total switching activity of the subsequent adder tree. This direct reduction in internal toggling is a primary contributor to the minimisation of dynamic power dissipation in low-power arithmetic circuits.

2.5 Partial Product Accumulation: Tree Compression

While Radix-4 Booth encoding effectively halves the number of partial product rows, accumulating the remaining 8 rows from a 16-bit operand using a linear ripple carry structure would still incur an unacceptable propagation delay. To break the $O(N)$ linear delay barrier, parallel reduction matrices, commonly known as tree compressors are employed to achieve a logarithmic $O(\log N)$ delay profile.

2.5.1 The Wallace Tree Architecture

Introduced by C. S. Wallace in 1964, the Wallace tree was one of the earliest parallel topologies designed to accelerate multiplication [11]. The fundamental principle of the Wallace tree is to process partial product bits in parallel columns without immediately resolving the carries. It groups the bits of the same significance into sets of three. These sets are then compressed using FA which reduce 3 bits to 2 bits: Sum and Carry. HAs are also used to align columns.

The Wallace algorithm mandates that the matrix be compressed as aggressively as possible at every single stage until only two rows remain. While this achieves the desired high-speed logarithmic delay, the aggressive compression strategy often results in an excessive use of Half Adders and Full Adders. This hardware overhead increases the silicon area and introduces complex, irregular wiring topologies that can exacerbate routing congestion and wiring capacitance during physical design.

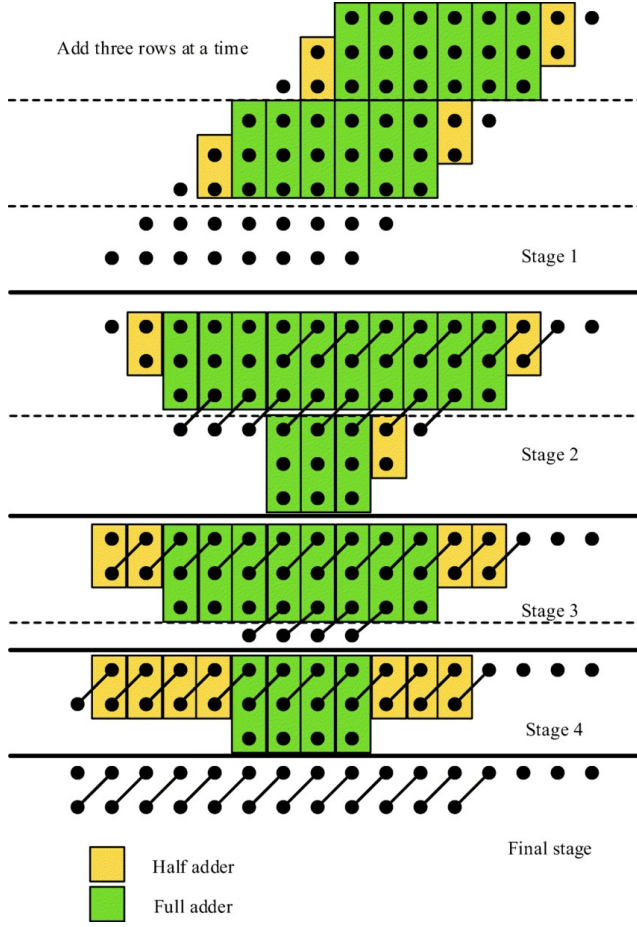


Figure 7: Dot Diagram of staged parallel reduction of 8-row partial products matrix using Wallace Reduction Tree

2.5.2 The Dadda Reduction Tree

To optimise the hardware utilisation of parallel multipliers, L. Dadda introduced a refined tree architecture in 1965 [12]. The Dadda tree operates on the exact same carry save principle as the Wallace tree but employs a strictly optimal compression strategy.

Instead of reducing the matrix as much as possible at every stage, the Dadda algorithm only deploys adders when the height of a specific column exceeds a predefined maximum for that stage. These maximum heights form a specific sequence, determined iteratively working backwards from the final two rows: 2, 3, 4, 6, 8, 13, 19, etc.

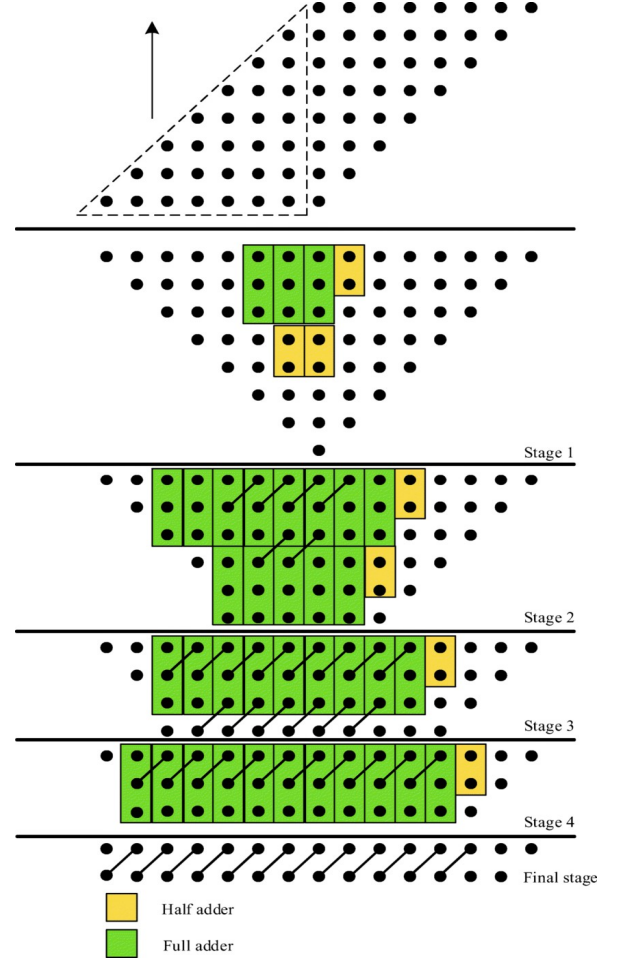


Figure 8: Dot Diagram of staged parallel reduction of 8-row partial products matrix using Dadda Reduction Tree.

2.5.3 Architectural Advantages of Dadda over Wallace

The primary distinction between the two architectures lies in their hardware efficiency. By adhering to the target height sequence, a Dadda tree guarantees the use of the absolute minimum number of FAs and HAs required to achieve the exact same propagation delay as a Wallace tree.

Because the Dadda architecture utilises fewer logic gates, it inherently possesses a smaller area footprint and more importantly for low power applications, fewer active gates directly translate to reduced parasitic capacitance and fewer internal switching events [13]. Consequently, the Dadda reduction tree provides a superior Power-Delay Product (PDP) compared to the Wallace tree, making it the optimal choice for the accumulation stage of an energy efficient multiplier.

2.6 Power Optimisation Strategies in Arithmetic Circuits

While architectural choices like Radix-4 Booth encoding and Dadda trees fundamentally reduce the logic depth and component count of a multiplier, realising a truly low power design requires targeted optimisations at both the logic and

standard cell levels. The strategies in managing dynamic power dissipation in advanced submicron technologies, such as the 22nm process is highly critical.

2.6.1 Sources of CMOS Power Dissipation

Power consumption in CMOS digital circuits is broadly categorised into static and dynamic power. Static power is primarily driven by subthreshold leakage currents, which have become increasingly prominent as transistor geometries shrink. However, in highly active datapath components like multipliers, dynamic power dissipation remains the dominant factor [11].

Dynamic power (P_{dyn}) is mathematically expressed as:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V_{\text{dd}}^2 \cdot f \quad (9)$$

where α is the switching activity factor, C is the physical load capacitance, V_{dd} is the supply voltage and f is the clock frequency. In unoptimised arithmetic circuits, a significant portion of α is caused by "glitching", spurious and unnecessary logic transitions that occur as signals propagate through unequal path lengths before reaching a steady state.

2.6.2 Operand Isolation and Clock Gating

To directly minimise the switching activity factor (α), system level data gating techniques such as Operand Isolation are highly effective [12]. In many DSP applications, the multiplier hardware may sit idle for several clock cycles while other operations execute. If the input buses to the multiplier are permitted to fluctuate during these idle periods, the internal logic gates of the multiplier will continue to toggle, consuming dynamic power.

Operand isolation mitigates this by inserting a layer of registers at the multiplier inputs, controlled by an explicit Enable (EN) signal. When EN is not asserted, the input registers hold their previous state, effectively freezing the combinational logic cloud of the multiplier and forcing α to near zero. Furthermore, modern synthesis tools can leverage this EN signal to infer Clock Gating cells, physically shutting off the clock tree to the isolated registers and yielding substantial additional power savings.

2.6.3 Logic Level Optimisation via AOI Gates

Beyond systemic isolation, the physical implementation of the logic gates themselves drastically influences the capacitance and delay of the circuit. Standard cell libraries offer complex logic gates, such as AND-OR-Invert (AOI) and OR-AND-Invert (OAI) cells, which compute multiple Boolean operations within a single, cohesive transistor structure [13].

Replacing discrete sequential AND and OR gates with combined AOI gates within the FAs and partial product generators reduces the total transistor count and eliminates intermediate routing nodes. This reduction in parasitic wiring capacitance and logic depth further suppresses glitching and minimises the overall energy required per multiplication operation.

Consequently, the most effective methodology for designing energy efficient arithmetic circuits combines the systemic reduction of partial products in Booth encoding, the equalisation of parallel path delays in Dadda reduction tree and the stringent suppression of switching activity in using Operand Isolation technique and AOI standard cells.

3 Design Methodology of the Optimised Multiplier

To achieve the stringent energy and performance targets required for modern digital signal processing, an optimised 16-bit signed multiplier was designed. The architecture replaces the conventional linear accumulation grid with a highly structured, parallel topology. This section details the behavioral Register Transfer Level (RTL) implementation of the proposed Radix-4 Booth-Dadda multiplier using Verilog Hardware Design Language (HDL), alongside the physical synthesis methodology employed to map the design to a 22nm CMOS technology node.

3.1 Architecture Overview

The proposed 16-bit multiplier is architecturally partitioned into three distinct, interconnected structural stages. This modular approach ensures synthesizability and allows for targeted logic-level optimisations at each phase of the multiplication operation. The three primary modules are:

1. **Partial Product Generation:** A Modified Radix-4 Booth Encoder that halves the required number of addition rows of partial products (PP1 $\rightarrow$ PP8).
2. **Parallel Accumulation:** A structurally instantiated Dadda reduction tree that compresses the rows with a logarithmic delay profile (8 rows $\rightarrow$ 2 rows).
3. **Final Addition:** A terminal Carry Propagate Adder (CPA) that resolves the final two rows (R1, R2) into the 32-bit product.

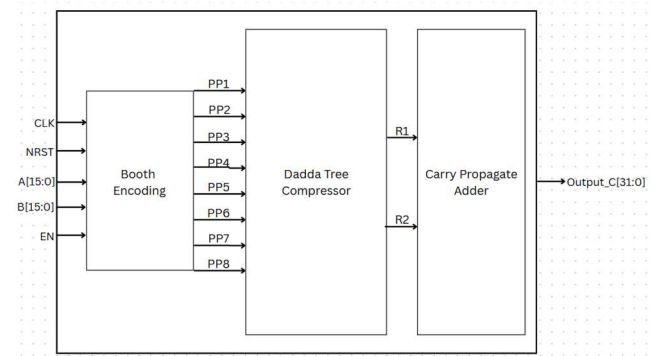


Figure 9: Top level architectural block diagram of the 16-bit low power multiplier

3.2 Modified Radix-4 Booth Encoder Implementation

The first phase of the design addresses the primary bottleneck of standard multipliers: the generation of 16 discrete partial product rows. The behavioural Verilog implementation of the Radix-4 Booth Encoder resolves this

by scanning the 16-bit multiplier operand in overlapping 3-bit windows.

To ensure mathematical accuracy for signed two's complement operations, the 16-bit multiplier is appended with an implicit zero at the least significant bit position before encoding begins as shown in Figure 6. Based on the Boolean value of each 3-bit window, the encoder logic selects the appropriate multiple of the multiplicand ($-2X$, $-X$, 0 , $+X$, $+2X$). Operations requiring multiplication by two are implemented via zero cost left shift wiring in the behavioral RTL code. Negative operations are executed by inverting the bits and adding a '1' at the accumulation stage which is two's complement operation.

By applying this encoding scheme, the initial 16 partial product rows are strictly reduced to exactly 8 rows, drastically cutting the workload for the subsequent adder tree.

3.3 Behavioural RTL Modelling of the Architecture

Unlike rigid structural netlists that manually instantiate individual logic gates, this project utilised synthesisable behavioural and dataflow modelling at the Register Transfer Level (RTL). This approach allows for a more flexible and highly optimisable design description, granting the synthesis engine the freedom to map the mathematical logic to the most efficient standard cells available in the technology library.

1. **Modified Radix-4 Booth Encoder:** This module was modelled using behavioural constructs (such as always blocks and case statements). The logic evaluates the 16-bit multiplier operand in overlapping 3-bit windows. Based on the evaluated case, the corresponding behavioural logic applies the necessary bitwise inversions and left shifts to generate the exactly eight partial product rows.
2. **Dadda Reduction Tree:** The parallel accumulation matrix was modelled using dataflow assignments, the continuous assign statements to represent the fundamental addition logic. The Verilog code was strictly organised to mirror the Dadda compression sequence ($8 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$ rows). By defining the Boolean logic of the sums and carries at each stage behaviourally, the synthesis tool was permitted to optimally map these equations into HAs, FAs and complex logic gates such as AOI gates.
3. **Final Addition Stage:** The terminal CPA was similarly modelled using dataflow logic to compute the final 32-bit accumulated product from the two remaining rows.

3.4 Logic Optimisation and Operand Isolation

To further suppress unnecessary switching activity during idle clock cycles, a system level data gating technique was integrated into the top level Verilog module. An external

Enable (EN) signal was coded to govern an array of input isolation registers. When EN is not asserted, the input operands are held in their previous state, effectively freezing the combinational logic cloud of the Booth Dadda tree and dropping the dynamic switching factor (α) to near zero.

3.5 VLSI Synthesis and Technology Mapping

Following functional verification via testbench simulation, the behavioral RTL design was advanced to physical logic synthesis. The primary objective of this phase is to translate the technology independent Verilog code into an optimised gate level netlist.

The synthesis was executed using the Synopsys Design Compiler, mapping the architecture strictly to the GlobalFoundries (GF) 22nm standard cell library. The compiler was constrained under typical Process, Voltage, and Temperature (PVT) conditions to simulate realistic hardware environments. As the architecture was described behaviourally, the Design Compiler's logic optimisation algorithms had the freedom to infer highly efficient AOI complex gates for the Dadda tree additions. Furthermore, the tool successfully interpreted the behavioural EN logic, automatically inferring native Clock Gating cells to enforce the systemic power reduction strategy.

The outputs of this synthesis flow specifically the Static Timing Analysis (STA), dynamic power estimation and total silicon area reports providing the quantitative metrics required for the comparative evaluation detailed in the next section.

4 Simulation and Performance Evaluation

4.1 Functional Verification Results

Prior to evaluating hardware efficiency, mathematical accuracy was verified. Both multipliers' designs were instantiated within a Verilog testbench environment and stimulated with an identical suite of test vectors, encompassing critical corner cases, alternating bit patterns, and randomised inputs.

Analysis of the simulation waveforms confirmed that both architectures successfully computed the correct 32-bit product for all signed two's complement inputs. Furthermore, the simulation verified the behaviour of the operand isolation logic in the Booth Dadda design, demonstrating that internal signal toggling successfully ceased when the Enable (EN) signal was de-asserted, confirming its readiness for power optimised logic synthesis.

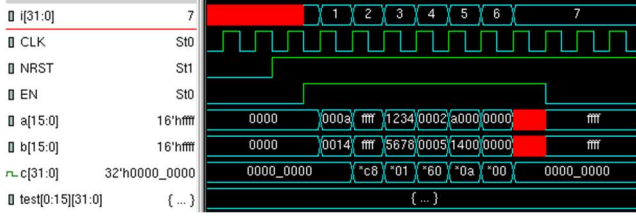


Figure 10: Functional simulation waveforms of 16-bit Booth Dadda multiplier showing operand isolation (EN)

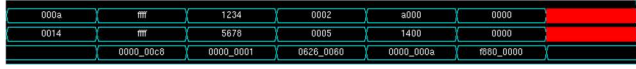


Figure 11: Zoomed-in functional simulation waveforms of 16-bit Booth Dadda multiplier showing signed multiplications

4.2 Synthesis Results and PPA Comparison

The verified behavioural RTL models were synthesised using the Synopsys Design Compiler, targeting the GlobalFoundries (GF) 22nm CMOS standard cell library. The extracted physical metrics, Critical Path Delay, Total Dynamic Power and Total Cell Area are summarised in the table below:

Table 5: PPA Comparison of different 16-bit Multiplier Architectures.

Architecture	Delay (ns)	Dynamic Power (μW)	Cell Area (unit/ $1\mu m^2$)
Array	2.45	62.32	501
Booth-Dadda	1.62	42.23	930
Improvement	-33.88%	-32.23%	+85.63%

4.3 Performance (Delay) Analysis

The static timing analysis reveals a substantial performance enhancement in the proposed architecture. The baseline array multiplier exhibits a critical path delay of 2.45ns. This high latency is directly attributable to its linear $O(N)$ accumulation grid, where the carry bits must propagate sequentially through 16 stages of adders.

In contrast, the Booth-Dadda multiplier achieves a critical path delay of just 1.62ns, representing a 33.88% reduction in computational latency. This acceleration is the direct result of the Dadda tree's logarithmic $O(\log N)$ compression sequence. By strictly compressing the partial products in parallel across only four structural stages, the data arrival time to the final Carry Propagate Adder is drastically shortened, allowing the circuit to comfortably meet higher clock frequency constraints.

4.4 Dynamic Power Dissipation Analysis

The most critical objective of this research was the optimisation of energy consumption. The synthesis reports

demonstrate that the baseline array multiplier consumes $62.32\mu W$ of total dynamic power. This high consumption is driven by excessive unnecessary logic transitions caused by unequal signal arrival times propagating down the linear adder chain.

The optimised Booth-Dadda architecture successfully reduces the total dynamic power to $42.23\mu W$, a highly significant reduction of 32.23%. This power saving is achieved through a multitiered approach:

- 1. Reduced Workload:** The Radix-4 Booth encoder halves the number of partial product rows from 16 to 8, immediately cutting the required switching activity factor (α) of the entire system.
- 2. Glitch Suppression:** The balanced, parallel nature of the Dadda tree ensures that signals arrive at the summation nodes more uniformly, drastically reducing spurious toggling and wasted capacitive charging.
- 3. Logic Optimisation:** The synthesis tool's inference of complex AOI gates within the Dadda matrix, combined with the explicitly coded operand isolation techniques, minimises parasitic capacitance and eliminates idle cycle switching.

4.5 Area Trade Off and Power-Delay Product (PDP)

As indicated in Table 5, the advanced logic of the Booth encoder and the complex wiring of the parallel Dadda tree require 930 area units, representing an 85% increase in silicon footprint compared to the highly regular 501 units of the array multiplier.

However, in modern submicron DSP applications, area is frequently traded for energy efficiency. The true measure of an arithmetic circuit's efficiency is its Power Delay Product (PDP), which represents the energy consumed per switching operation.

Array Multiplier PDP:

$$62.32\mu W \times 2.45ns = 152.68fJ \quad (10)$$

Booth-Dadda PDP:

$$42.23\mu W \times 1.62ns = 68.41fJ \quad (11)$$

where fJ is femtojoules, $1f = 10^{-15}$.

Despite the larger physical footprint, the proposed Booth-Dadda multiplier requires 55.2% less energy per calculation than the conventional array. This vastly superior energy delay profile conclusively justifies the area overhead and validates the architecture as an optimal solution for power constrained, high performance environments.

5 Conclusion and Recommendations for Future Work

5.1 Conclusion

The proliferation of portable digital signal processors and wireless communication systems has driven an urgent need for highly energy efficient arithmetic hardware. This is because multiplication is a fundamentally power intensive and frequently invoked operation. Therefore, optimising the multiplier's datapath is critical for the thermal management and battery longevity of modern VLSI systems.

This paper presented the design, implementation, and rigorous performance evaluation of an optimised 16-bit signed CMOS multiplier. By moving away from the conventional, linear accumulation grid, the proposed architecture successfully synergised a Modified Radix-4 Booth encoder with a parallel Dadda reduction tree. This architectural combination was further enhanced through logic level optimisations, including operand isolation and the inference of complex AOI standard cells.

Synthesised using the GlobalFoundries 22nm technology node, the proposed Booth-Dadda architecture demonstrated vastly superior performance characteristics compared to a standard 16-bit array multiplier baseline. The parallel reduction matrix successfully broke the linear carry propagation chain, yielding a 33.88% reduction in critical path delay. Concurrently, the halving of the partial product rows and the subsequent equalisation of signal arrival times severely suppressed internal glitching, resulting in a 32.23% reduction in total dynamic power dissipation.

While the complex routing of the Dadda tree and the encoding logic resulted in an 85% increase in total silicon area, the resulting 55.2% improvement in the Power-Delay Product (PDP) conclusively justifies this hardware trade off. Ultimately, the results validate that the Radix-4 Booth-Dadda architecture provides a highly robust and energy efficient solution, making it exceptionally well suited for integration into the next generation of high performance, low power digital signal processing architectures.

5.2 Recommendations for future work

Future work should focus on post-layout physical design and timing simulations to further verify the architecture against parasitic wiring capacitances and varying thermal corners.

Besides, further architectural refinements can be explored within the parallel accumulation matrix. Specifically, 4 to 2 compressors could also be evaluated in the Dadda Tree compressor to replace the current tiers Has and FAs with the hope of reducing hardware complexity and further reducing delay.

Finally, future work should investigate hybrid methods to reduce the silicon area while maintaining power and performance gains as the current Booth-Dadda design requires an 85.63% increase compared to conventional array multiplier. This could be minimised by exploring techniques such as approximate computing in approximating logic substitution.

Acknowledgements

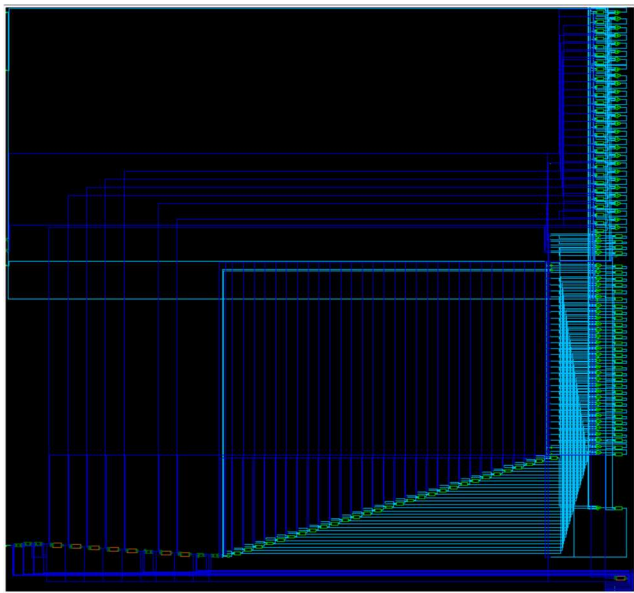
I would like to express my deepest gratitude to my supervisor, Prof. Gwee Bah Hwee for his patience, continuous guidance and support throughout this Undergraduate Research Opportunity Programme (UROP). His expertise and insightful feedback were instrumental in shaping this research. I am truly grateful for the opportunity to learn and work under his mentorship.

References

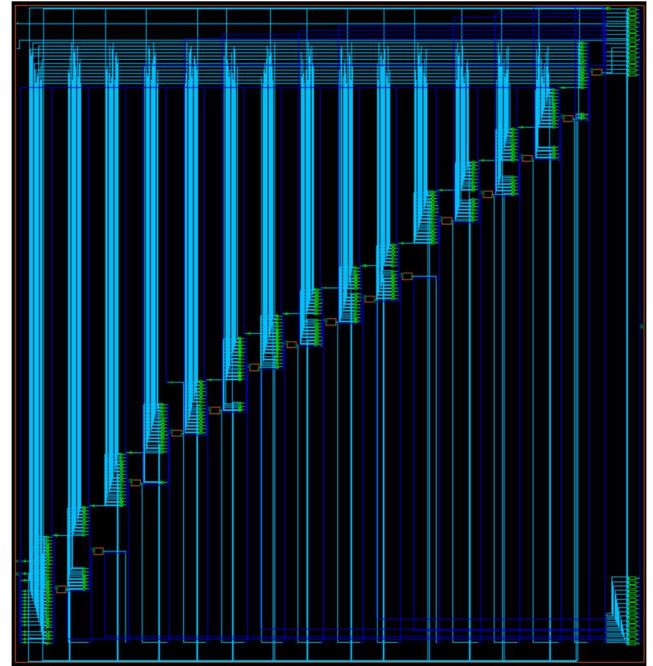
- [1] R. P. Brent and H. T. Kung, "A regular layout for parallel adders," *IEEE Transactions on Computers*, vol. C-31, no. 3, pp. 260-264, Mar. 1982.
- [2] A. Bellaouar and M. I. Elmasry, *Low-Power Digital VLSI Design: Circuits and Systems*. Norwell, MA, USA: Kluwer Academic Publishers, 1995.
- [3] A. D. Booth, "A signed binary multiplication technique," *The Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236-240, 1951.
- [4] L. Dadda, "Some schemes for parallel multipliers," *Alta Frequenza*, vol. 34, pp. 349-356, Mar. 1965.
- [5] J. M. Rabaey, A. Chandrakasan, and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Education, 2003.
- [6] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Addison-Wesley, 2010.
- [7] V. G. Oklobdzija, "High-speed VLSI arithmetic units: Adders and multipliers," in *Design of High-Performance Microprocessor Circuits*, Piscataway, NJ, USA: IEEE Press, 2001, pp. 241-267.
- [8] S. Kuang, J. Wang, and C. Guo, "Modified multipliers for DSP applications," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 18, no. 12, pp. 1732-1736, 2010.
- [9] A. P. Chandrakasan and R. W. Brodersen, "Minimizing power consumption in digital CMOS circuits," *Proceedings of the IEEE*, vol. 83, no. 4, pp. 498-523, 1995.
- [10] O. L. MacSorley, "High-speed arithmetic in binary computers," *Proceedings of the IRE*, vol. 49, no. 1, pp. 67-91, 1961.

- [11] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Transactions on Electronic Computers*, vol. EC-13, no. 1, pp. 14-17, 1964.
- [12] L. Dadda, "Some schemes for parallel multipliers," *Alta Frequenza*, vol. 34, pp. 349-356, 1965.
- [13] W. J. Townsend, E. E. Swartzlander, and J. A. Abraham, "A comparison of Dadda and Wallace multiplier delays," in *Advanced Signal Processing Algorithms, Architectures, and Implementations XIII*, SPIE, vol. 5205, pp. 552-560, 2003.

Appendix



Appendix 1: Schematic of the Booth-Dadda Multiplier



Appendix 2: Schematic of the Array Multiplier